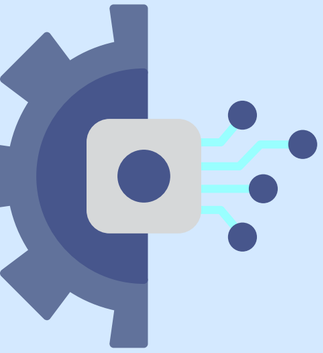


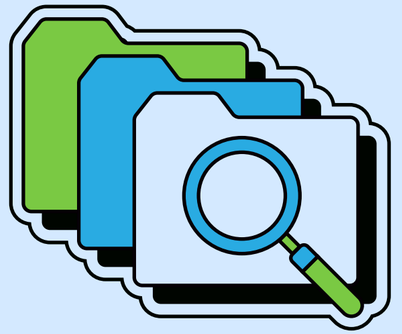


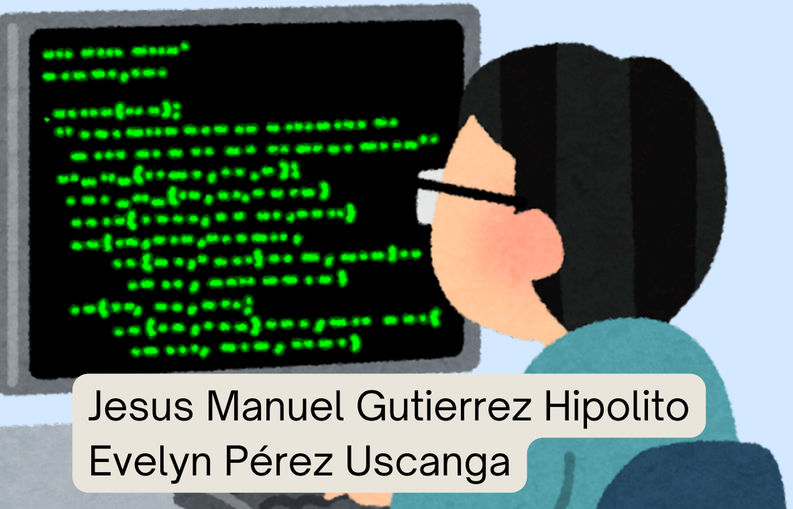


Colecciones en java

ArrayList

Programación Orientada a Objetos

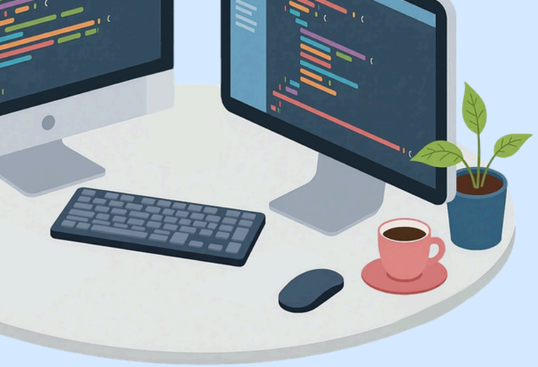




Jesus Manuel Gutierrez Hipolito
Evelyn Pérez Uscanga





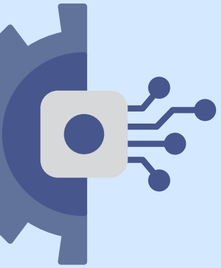






¿Qué es?

Es una clase de la biblioteca estándar (java.util) que implementa la interfaz List mediante un arreglo interno redimensionable, permitiendo almacenar colecciones ordenadas de objetos con acceso posicional eficiente. Guarda sus elementos en un arreglo interno que crece cuando hace falta.





Dinámico

Cambia de tamaño solo: no hay que fijar su longitud de antemano.

Ordenado

Conserva el orden de inserción y se accede por índice: 0, 1, 2...

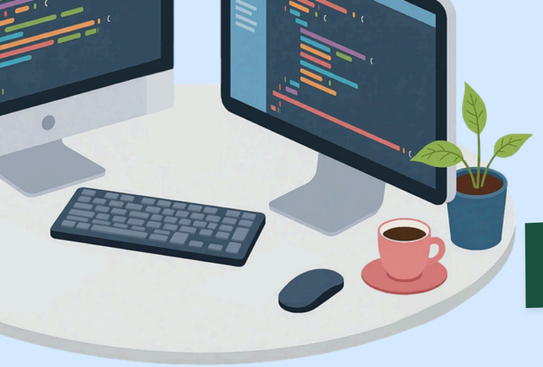
Flexible

Admite elementos repetidos y null, pero solo objetos como String o Integer.













Dónde encaja en las colecciones

Iterable<E>

Permite recorrer con for-each

Collection<E>

Operaciones básicas: add, remove, size

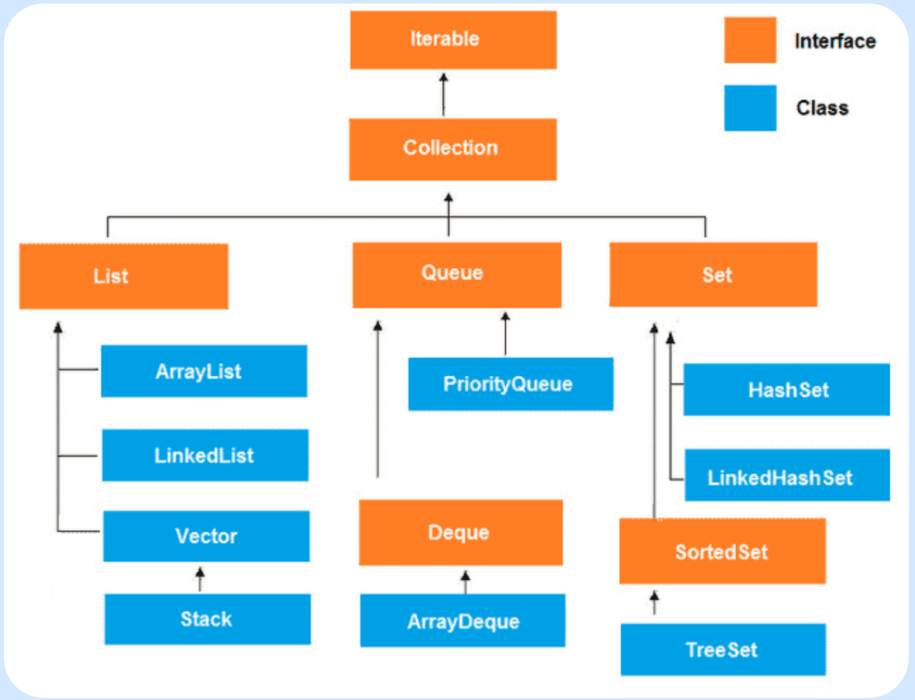
List<E>

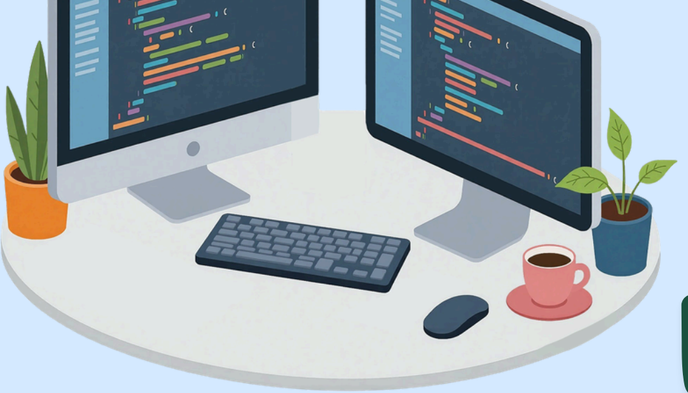
Elementos ordenados con índice

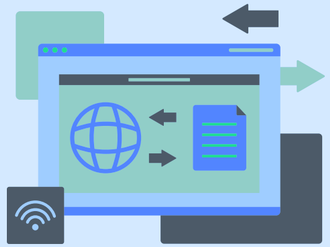
ArrayList<E>

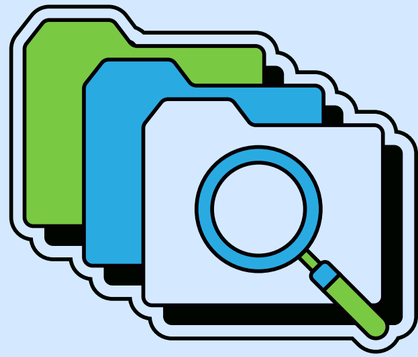
Implementación con arreglo redimensionable

ArrayList extiende AbstractList e implementa también RandomAccess, Cloneable y Serializable. Sus "primos" en el framework son LinkedList y Vector.









Crear un ArrayList

```java
import java.util.ArrayList;

ArrayList<String> frutas = new ArrayList<>();
ArrayList<Integer> nums = new ArrayList<>(50);
List<String> copia = new ArrayList<>(frutas);
```

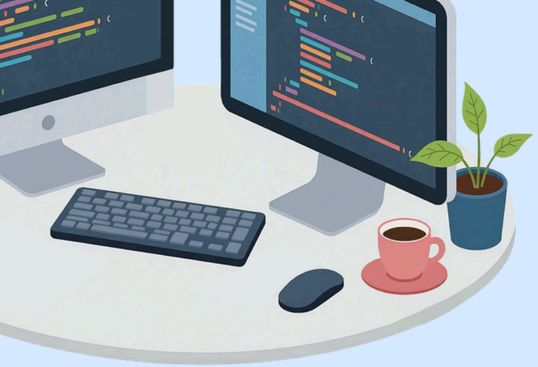
Lista vacía. La capacidad inicial es 10, que se reserva al primer add.

Capacidad inicial de 50: evita crecer varias veces.

Copia los elementos de otra colección.

Los tipos primitivos usan su clase envoltorio: Integer, Double, Boolean...



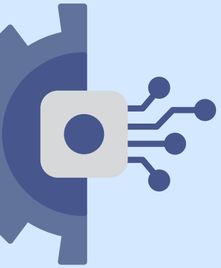






Operaciónes Principales

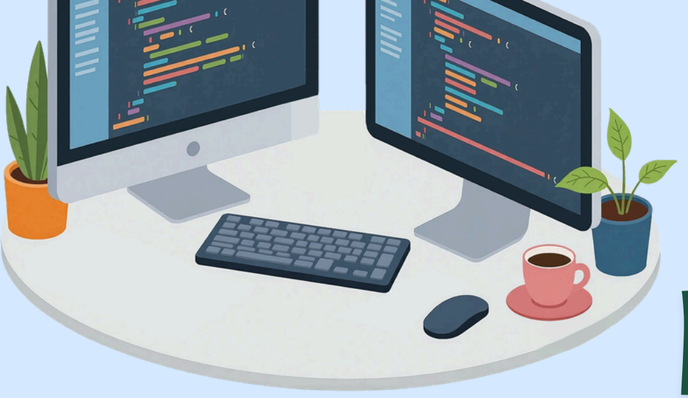
OPERACIÓN	CÓDIGO
Agregar al final	`frutas.add("Pera");`
Insertar en una posición	`frutas.add(0, "Kiwi");`
Leer un elemento	`frutas.get(1);`
Reemplazar	`frutas.set(1, "Uva");`
Eliminar por índice	`frutas.remove(0);`
Buscar un elemento	`frutas.contains("Pera");`
Cantidad de elementos	`frutas.size();`
Vaciar la lista	`frutas.clear();`

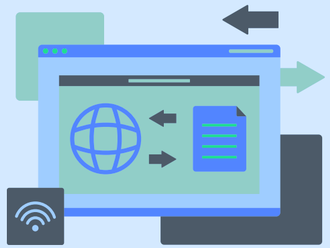


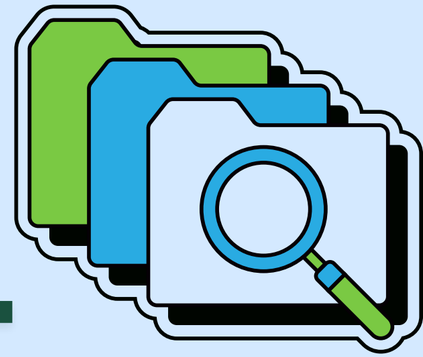




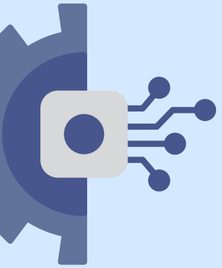








Recorrer un ArrayList



```java
for (int i = 0; i < frutas.size(); i++) {
  System.out.println(frutas.get(i));
}
```

for con índice

Útil cuando necesitas la posición.

```java
for (String f : frutas) {
  System.out.println(f);
}
```

for-each

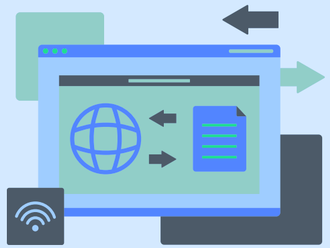
La forma más legible.

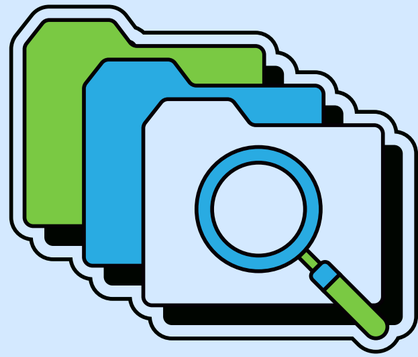
```java
frutas.forEach(f -> System.out.println(f));
```

forEach y lambda

Estilo funcional, desde Java 8.

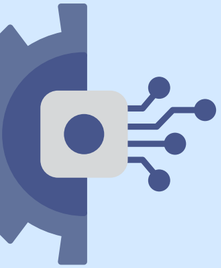




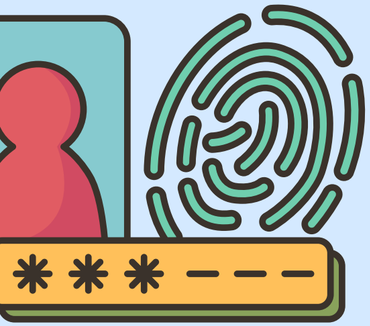


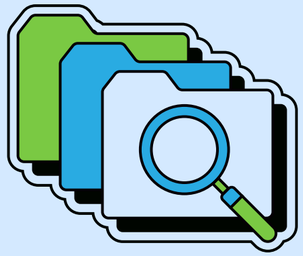
Ejemplo

```java
import java.util.ArrayList;
public class Main {
  public static void main(String[] args) {
    ArrayList<String> tareas = new ArrayList<>();
    tareas.add("Estudiar");      [Estudiar]
    tareas.add("Practicar");     [Estudiar, Practicar]
    tareas.add(1, "Descansar");  [Estudiar, Descansar, Practicar]
    tareas.remove("Practicar");  [Estudiar, Descansar]
    for (String t : tareas) {
      System.out.println(t);
    }
  }
}
```

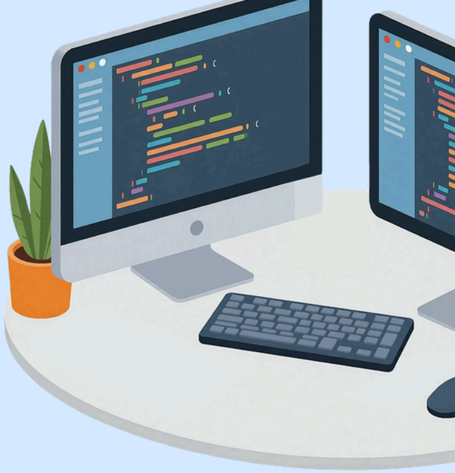




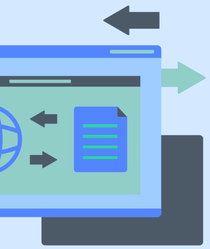


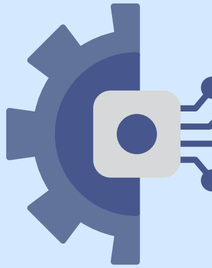


Caracteristicas



- Redimensionamiento dinámico: A diferencia de los arreglos nativos (**int[]** o **String[]**), cuyo tamaño es estático e inmutable tras la inicialización, **ArrayList** incrementa o reduce su capacidad automáticamente conforme se agregan o eliminan elementos.



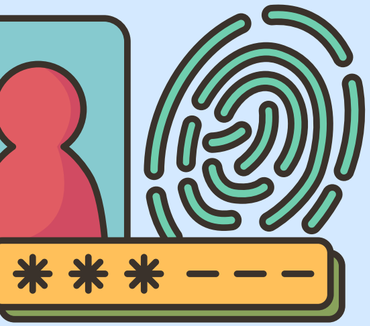


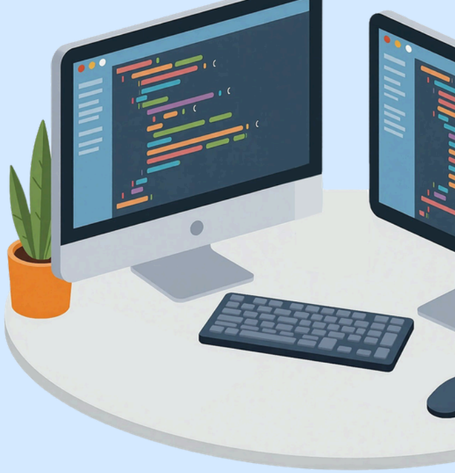
- Acceso aleatorio de tiempo constante: Al implementar la interfaz marcadora **RandomAccess** y basarse en bloques de memoria contigua, operaciones como **get(int index)** y **set(int index, E element)** acceden a cualquier posición de forma instantánea mediante cálculo de desplazamiento de punteros.



- Mantenimiento del orden y duplicados: Preserva el orden cronológico de inserción, admite valores repetidos y tolera múltiples referencias null.

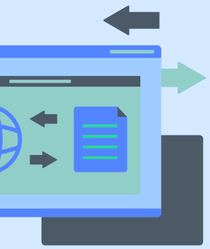


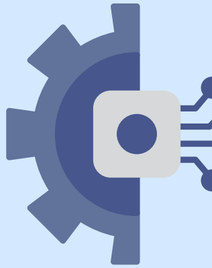




Caracteristicas

- Ausencia de sincronización (Not Thread-Safe): Por diseño, sus métodos no están sincronizados para evitar sobrecostes de rendimiento en ejecuciones monohilo. En aplicaciones concurrentes, debe protegerse externamente mediante Collections.synchronizedList() o reemplazarse por estructuras de java.util.concurrent como CopyOnWriteArrayList.

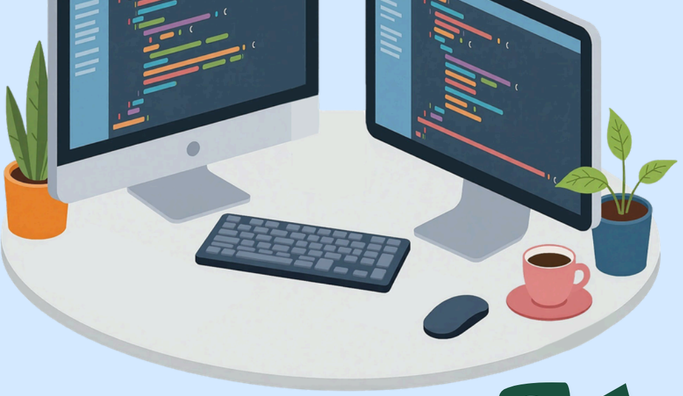


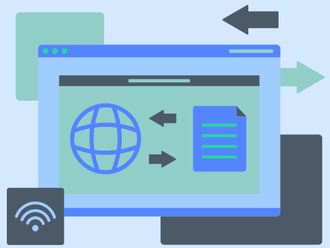


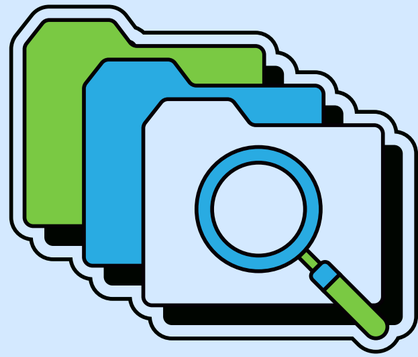
- Soporte exclusivo para tipos por referencia: No puede almacenar tipos de datos primitivos (**int**, **double**, **boolean**, **char**). Requiere el uso de sus clases envolventes (wrapper classes como **Integer**, **Double**, **Boolean**), aprovechando las capacidades de autoboxing y unboxing del compilador.



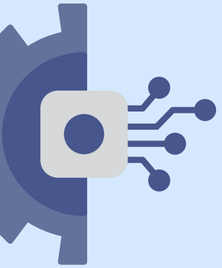








Cómo crece por dentro



Arreglo interno lleno: capacidad 10



Nuevo arreglo: capacidad 15 (10 casillas ocupadas, 5 libres)



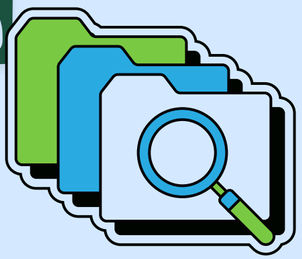
nueva = vieja + vieja / 2

Crece cerca de 1,5 veces. Copiar cuesta O(n), pero agregar al final es O(1) amortizado.

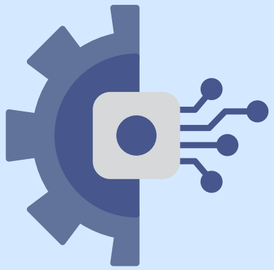
Mecánica interna y gestión de capacidad







- *Capacidad inicial*: Al instanciar mediante el constructor sin argumentos (**new ArrayList<>()**), la lista inicializa un arreglo interno vacío. En el momento en que se añade el primer elemento vía **add()**, el arreglo se inicializa con una capacidad por defecto de 10 elementos.





- *Estrategia de crecimiento*: Cuando la cantidad de elementos excede la capacidad actual, la máquina virtual crea un nuevo arreglo cuyo tamaño es aproximadamente un 50% mayor que el anterior **(nuevaCapacidad = viejaCapacidad + (viejaCapacidad >> 1)**).

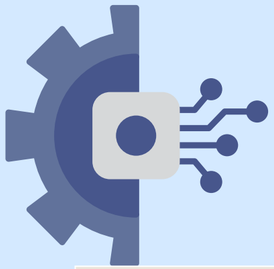


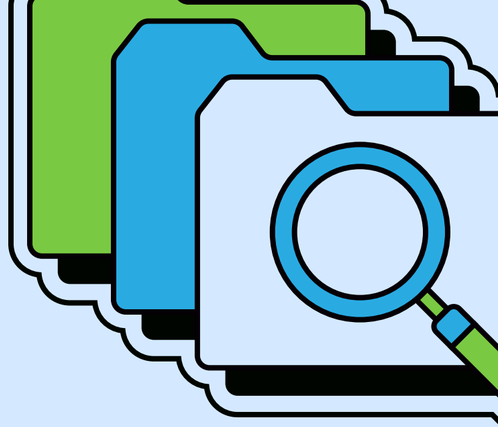
- *Copia en memoria*: Los datos existentes se transfieren al nuevo bloque de memoria empleando el método nativo optimizado **System.arraycopy()**, descartando el arreglo previo para su recolección por el recolector de basura.



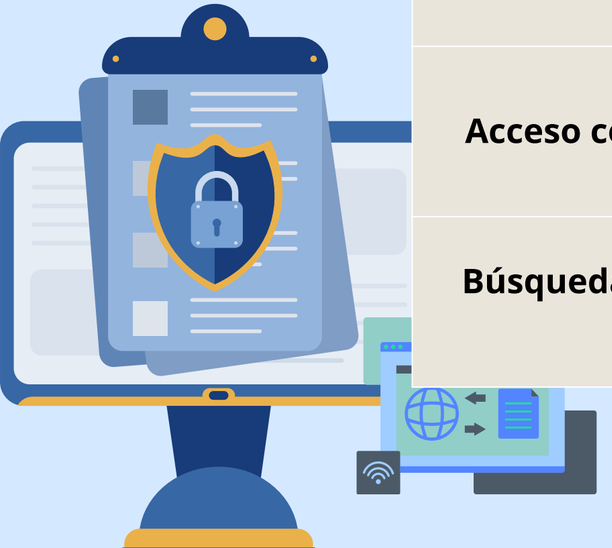


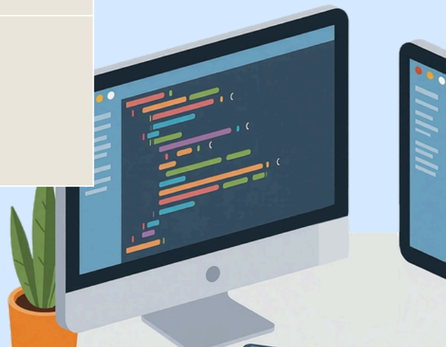
Cuándo evitar ArrayList



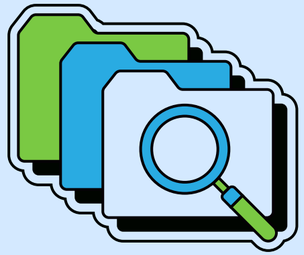


Requisito del sistema	Problema con ArrayList	Alternativa recomendada
Estructuras FIFO (Colas de tareas)	Extraer o insertar al inicio (remove(0) / add(0, e)) desplaza todos los elementos existentes.	ArrayDeque.
Cálculo numérico masivo	El *autoboxing* (convertir int a Integer) multiplica el consumo de memoria por 4 o 5 veces y genera presión en el GC.	Arreglos primitivos (double[], int[]).
Acceso concurrente multihilo	Riesgo de ConcurrentModificationException y corrupción de punteros internos.	CopyOnWriteArrayList (para lecturas masivas) o ConcurrentLinkedQueue.
Búsqueda frecuente por valor (contains)	Requiere un recorrido lineal evaluando equals() elemento por elemento.	HashSet o HashMap.







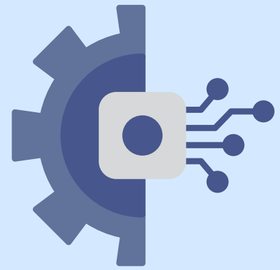




Escenario posible de aplicación

- Procesamiento por lotes y agregadores -





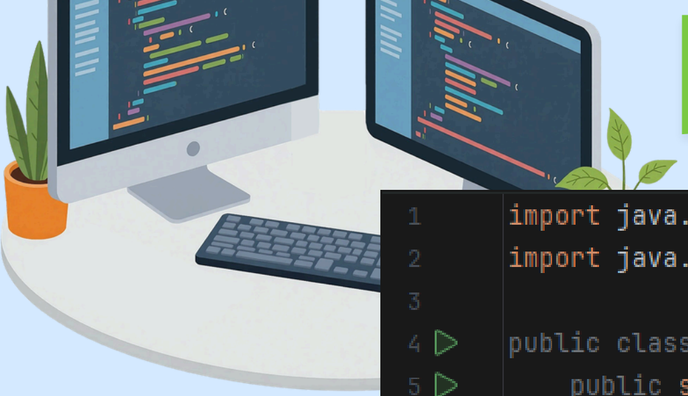
Caso de uso: Lectura de archivos masivos (CSV, logs, mensajes de colas) donde se acumulan elementos en fragmentos de tamaño fijo antes de disparar un volcado a la base de datos o un llamado HTTP.

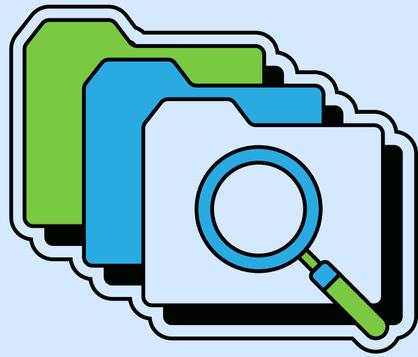
Por qué ArrayList: Minimiza la sobrecarga de memoria por elemento frente a alternativas como LinkedList. Al reservar la capacidad exacta de cada lote (new ArrayList<>(batchSize)), el uso de memoria es predecible y la recolección de basura (Garbage Collection) tiene menos objetos de cabecera que rastrear.





Ejemplo de código





```java
import java.util.ArrayList;
import java.util.List;

public class EjemploArrayList {
    public static void main(String[] args) {
        // Inicialización usando polimorfismo con la interfaz List
        List<String> servidores = new ArrayList<>();

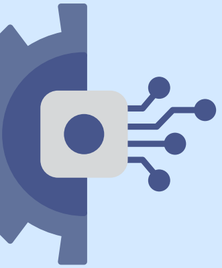
        // Inserción de elementos
        servidores.add("Web-01");
        servidores.add("Database-01");
        servidores.add("Cache-01");

        // Acceso por índice
        String principal = servidores.get(0);

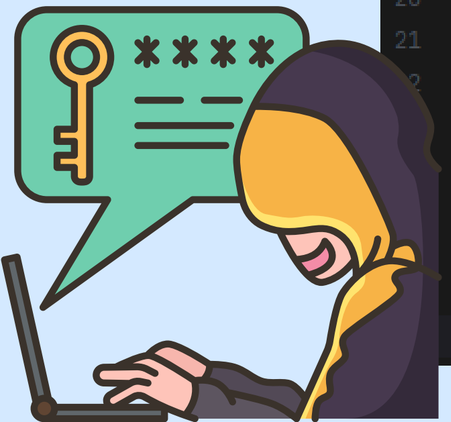
        // Modificación
        servidores.set(1, "Database-Primary");

        // Eliminación por índice o por objeto
        servidores.remove( o: "Cache-01");

        // Iteración idiomática
        for (String servidor : servidores) {
            System.out.println("Nodo activo: " + servidor);
        }
    }
}
```

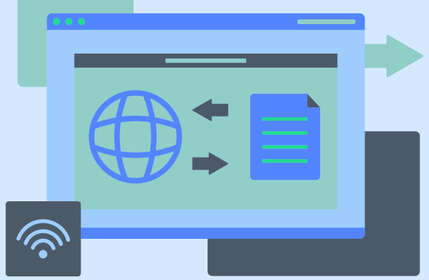




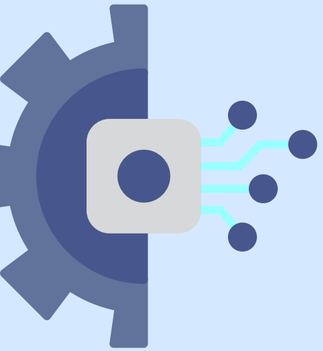












Preguntas?

